Buổi 6

Danh sách và Danh sách đặc

Học phần: Cấu trúc dữ liệu và Giải thuật

Nội dung bài học

1. Khái niệm danh sách

1.1 Danh sách là gì?
1.2 Các phép toán trên danh sách

2. Danh sách đặc

2.1 Định nghĩa và biểu diễn danh sách đặc
2.2 Khởi tạo, tạo mới và duyệt danh sách đặc
2.3 Tìm kiếm trên danh sách đặc
2.4 Thêm một phần tử vào danh sách đặc
2.5 Thêm vào danh sách có thứ tự
2.6 Hủy phần tử khỏi danh sách đặc
2.7 Hủy nhiều phần tử – lỗi thường gặp và cách làm đúng
2.8 Sắp xếp, tách, nhập danh sách đặc
2.9 Ưu nhược điểm và ứng dụng của danh sách đặc
2.10 Mảng động tự mở rộng – khắc phục giới hạn MaxLen
2.11 Ví dụ tổng hợp: quản lý điểm SV bằng danh sách đặc

3. Tổng kết và thực hành

3.1 Bài thực hành buổi 6 – Thư viện thao tác danh sách đặc
3.2 Câu hỏi và bài tập

1.1 Danh sách là gì?

Định nghĩa

Danh sách là một tập hợp **có thứ tự** gồm các phần tử cùng kiểu: $L = (a_1, a_2, \ldots, a_n)$. n là **chiều dài** danh sách (n = 0: danh sách **rỗng**). a_1 là phần tử **đầu**, a_n là phần tử **cuối**; mỗi phần tử (trừ cuối) có một phần tử **kế tiếp**.

- **Ví dụ thực tế:** danh sách SV của lớp, hàng đợi khách hàng, danh sách bài hát, các dòng văn bản trong trình soạn thảo, đa thức (danh sách các số hạng).
- **Phân loại theo cách lưu trữ:**

Loại	Cách lưu trữ	Buổi
Danh sách đặc	Các phần tử đặt **liên tiếp** trong bộ nhớ (mảng)	6
Danh sách liên kết	Các phần tử nằm rải rác, nối với nhau bằng **con trỏ**	7, 8
Danh sách hạn chế	Chỉ thêm/bớt ở đầu/cuối: ngăn xếp, hàng đợi	9

Ví dụ 6.1 – Danh sách SV lớp Tin K24 (theo thứ tự tên)

L = (An, Bình, Chi, Dũng, Hoa), chiều dài 5.

- Phần tử đầu: An; phần tử cuối: Hoa.
- Phần tử kế tiếp của Chi là Dũng; của Hoa là **không có**.
- Thêm "Cường" mà vẫn giữ thứ tự tên ⇒ chèn giữa Chi và Dũng ⇒ L = (An, Bình, Chi, Cường, Dũng, Hoa).

- Danh sách là một **kiểu dữ liệu trừu tượng** (ADT): định nghĩa bởi tập phép toán; có thể cài đặt bằng mảng hoặc con trỏ.

1.2 Các phép toán trên danh sách

Phép toán	Ý nghĩa
Khởi tạo (Initialize)	Tạo danh sách rỗng
Tạo mới / nhập danh sách	Đưa dữ liệu ban đầu vào danh sách
Duyệt (Traverse)	Đi qua mọi phần tử theo thứ tự (để in, đếm, tính tổng…)
Tìm kiếm (Search)	Tìm phần tử thỏa điều kiện (theo khóa)
Thêm (Insert)	Thêm phần tử vào đầu, cuối, hoặc vị trí bất kỳ
Hủy / xóa (Delete)	Xóa phần tử đầu, cuối, tại vị trí, hoặc theo khóa
Sắp xếp (Sort)	Sắp lại các phần tử theo khóa
Tách (Split)	Chia một danh sách thành nhiều danh sách con theo một tiêu thức; tổng chiều dài không đổi
Nhập (Concat / Merge)	Ghép nối đuôi hoặc trộn xen kẽ theo thứ tự nhiều danh sách thành một
Sao chép (Copy)	Tạo bản sao giống hệt
Hủy danh sách (Clear)	Xóa toàn bộ phần tử (và giải phóng bộ nhớ nếu có)

Ví dụ 6.2 – Tách và nhập danh sách

- **Tách** L = (3, 8, 1, 6, 5, 2) theo tiêu thức chẵn/lẻ ⇒ L1 = (8, 6, 2), L2 = (3, 1, 5); 3 + 3 = 6 = chiều dài L.
- **Nhập** bằng ghép nối: (8, 6, 2) + (3, 1, 5) = (8, 6, 2, 3, 1, 5).
- **Nhập** bằng trộn có thứ tự (hai danh sách đã sắp): (2, 6, 8) và (1, 3, 5) ⇒ (1, 2, 3, 5, 6, 8) – thao tác **trộn** ở buổi 5.

Định nghĩa

Danh sách đặc (Condensed List) là danh sách mà không gian bộ nhớ lưu trữ các phần tử được đặt **liên tiếp nhau** trong bộ nhớ.

- **Biểu diễn:** dùng **mảng** các phần tử; cần biết trước **chiều dài tối đa** (MaxLen) và quản lý **chiều dài thực** Length.

```
const int MaxLen = 10000;      // hoặc: #define MaxLen 10000
int Length;
T CD_LIST[MaxLen];             // hoặc: T *CD_LIST = new
T[MaxLen];
```

- Nếu cấp phát động phải **kiểm tra** cấp phát thành công (CD_LIST != NULL).
- Có thể gom vào struct cho gọn:

```
typedef struct {
   T   M[MaxLen];
   int Length;
} CD_Type;
```

Ví dụ 6.3 – Hình ảnh bộ nhớ (T = int, 4 byte, địa chỉ bắt đầu 1000)

Chỉ số	0	1	2	3	4	5	…	9999
Địa chỉ	1000	1004	1008	1012	1016	1020	…	40996
Giá trị	15	10	20	18	40	?	…	?

- Length = 5: chỉ M[0..4] có nghĩa; M[5..9999] là vùng **dự trữ**.
- Địa chỉ phần tử thứ i = **1000 + 4·i** ⇒ truy cập trực tiếp **O(1)**.
- Bộ nhớ chiếm cố định 40.000 byte dù chỉ dùng 20 byte ⇒ **lãng phí** khi danh sách ngắn.

2.2 Khởi tạo, tạo mới và duyệt danh sách đặc

- **Khởi tạo:** chỉ cần cho chiều dài về 0.

```
void CD_Initialize(int &Len)
{  Len = 0;
   return;
}
```

- **Tạo mới / nhập danh sách:**

```
int CD_Create_List(T M[], int &Len)
{  if (Len > MaxLen)
      Len = MaxLen;
   for (int i = 0; i < Len; i++)
      M[i] = Input_One_Element();
   return (Len);
}
```

- Input_One_Element: hàm nhập một phần tử kiểu T – viết tùy theo T cụ thể.

- **Duyệt danh sách** – khung chung cho mọi thao tác "đi qua từng phần tử":

```
void CD_Traverse(T M[], int Len)
{  for (int i = 0; i < Len; i++)
      Process(M[i]);       // in, đếm, cộng dồn…
}
```

Ví dụ 6.4 – Áp dụng duyệt

```
float CD_Average(int M[], int Len)   // trung bình cộng
{  if (Len == 0) return 0;
   long S = 0;
   for (int i = 0; i < Len; i++) S += M[i];
   return (float)S / Len;
}
int CD_CountEven(int M[], int Len)   // đếm số chẵn
{  int c = 0;
   for (int i = 0; i < Len; i++)
      if (M[i] % 2 == 0) c++;
   return c;
}
```

- Mọi thao tác duyệt có độ phức tạp **O(Length)**.

- Áp dụng trực tiếp các thuật toán **buổi 2**: tìm tuyến tính (mọi danh sách), tìm nhị phân (danh sách **có thứ tự**).

```
int CD_Search(T M[], int Len, T X)
{  for (int i = 0; i < Len; i++)
      if (M[i] == X)
         return (i);
   return (-1);
}
// Danh sách có thứ tự tăng:
int CD_BinarySearch(T M[], int Len, T X)
{  int First = 0, Last = Len - 1;
   while (First <= Last)
   {  int Mid = (First + Last)/2;
      if (X == M[Mid]) return Mid;
      if (X < M[Mid]) Last = Mid - 1;
      else First = Mid + 1;
   }
   return (-1);
}
```

Ví dụ 6.5 – Tìm SV theo mã trong danh sách đặc các struct

```
typedef struct { char MaSV[11]; char HoTen[31]; float DTB;
} SinhVien;
int CD_SearchSV(SinhVien M[], int Len, char *Ma)
{  for (int i = 0; i < Len; i++)
      if (strcmp(M[i].MaSV, Ma) == 0)
         return i;
   return -1;
}
```

- So sánh chuỗi phải dùng **strcmp**, không dùng `==` (sẽ so sánh địa chỉ).

- **Ưu thế của danh sách đặc:** truy cập M[Mid] trong O(1) nên **tìm nhị phân** khả thi – danh sách liên kết thì không.

2.4 Thêm một phần tử vào danh sách đặc

- **Bài toán:** thêm phần tử có giá trị **NewValue** vào danh sách M có chiều dài Length tại vị trí **InsPos** (0 ≤ InsPos ≤ Length).

```
B1: IF (Length = MaxLen)
      Thực hiện Bkt                  // danh sách đầy
// Dời các phần tử từ InsPos..Length-1 ra sau một vị trí
B2: Pos = Length + 1
B3: IF (Pos = InsPos) Thực hiện B7
B4: M[Pos] = M[Pos-1]
B5: Pos--
B6: Lặp lại B3
B7: M[InsPos] = NewValue
B8: Length++
Bkt: Kết thúc
```

```
int CD_Insert_Element(T M[], int &Len, T NewValue, int
InsPos)
{  if (Len == MaxLen || InsPos < 0 || InsPos > Len)
      return (-1);
   for (int i = Len; i > InsPos; i--)
      M[i] = M[i-1];
   M[InsPos] = NewValue;
   Len++;
   return (InsPos);
}
```

Ví dụ 6.6 – Thêm 25 vào vị trí 2 của M = (15, 10, 20, 18, 40), Length = 5

Bước	Thao tác	M
Ban đầu		15 10 20 18 40 _
i = 5	M[5] = M[4]	15 10 20 18 40 **40**
i = 4	M[4] = M[3]	15 10 20 18 **18** 40
i = 3	M[3] = M[2]	15 10 20 **20** 18 40
Chèn	M[2] = 25	15 10 **25** 20 18 40

- Length = 6. Phải **dời 3** phần tử (Length − InsPos).
- **Lưu ý:** dời **từ cuối về** (i giảm); nếu dời từ InsPos về sau sẽ **ghi đè** mất dữ liệu.

- **Phân tích:** thêm cuối O(1); thêm đầu dời Length phần tử ⇒ **O(N)**.

2.5 Thêm vào danh sách có thứ tự

- Danh sách tăng dần, cần thêm X mà **vẫn giữ thứ tự**: tìm vị trí chèn (phần tử đầu tiên > X) rồi gọi CD_Insert_Element.

```
int CD_Insert_Order(T M[], int &Len, T X)
{  if (Len == MaxLen) return -1;
   int i = Len - 1;
   while (i >= 0 && M[i] > X)   // vừa tìm vừa dời
   {  M[i + 1] = M[i];
      i--;
   }
   M[i + 1] = X;
   Len++;
   return i + 1;
}
```

- Chính là **một bước của Insertion Sort** (buổi 4). Có thể dùng tìm nhị phân để tìm vị trí, nhưng số lần dời không đổi.

Ví dụ 6.7 – Lần lượt thêm 30, 12, 45, 12, 27 vào danh sách rỗng

Thêm	Danh sách sau khi thêm	Số lần dời
30	30	0
12	12 30	1
45	12 30 45	0
12	12 **12** 30 45	2 (phần tử mới đứng **sau** 12 cũ – ổn định)
27	12 12 **27** 30 45	2

- Xây dựng danh sách có thứ tự từ N phần tử bằng cách này tốn **$O(N^2)$** ở trường hợp xấu nhất.

2.6 Hủy phần tử khỏi danh sách đặc

- **Hủy tại vị trí DelPos:** dời các phần tử phía sau **lên trước** một vị trí, giảm Length.

```
B1: IF (Length = 0 OR DelPos ≥ Length)
      Thực hiện Bkt
B2: Pos = DelPos
B3: IF (Pos = Length - 1) Thực hiện B7
B4: M[Pos] = M[Pos+1]
B5: Pos++
B6: Lặp lại B3
B7: Length--
Bkt: Kết thúc
```

```
int CD_Delete_Element(T M[], int &Len, int DelPos)
{  if (Len == 0 || DelPos < 0 || DelPos >= Len)
      return (0);
   for (int i = DelPos; i < Len - 1; i++)
      M[i] = M[i+1];
   Len--;
   return (1);
}
```

Ví dụ 6.8 – Hủy phần tử tại vị trí 1 của M = (15, 10, 25, 20, 18, 40)

Bước	Thao tác	M (Length)
Ban đầu		15 10 25 20 18 40 (6)
i = 1	M[1] = M[2]	15 **25** 25 20 18 40
i = 2	M[2] = M[3]	15 25 **20** 20 18 40
i = 3	M[3] = M[4]	15 25 20 **18** 18 40
i = 4	M[4] = M[5]	15 25 20 18 **40** 40
Kết thúc	Length = 5	15 25 20 18 40

- M[5] vẫn còn giá trị 40 nhưng **nằm ngoài** danh sách (Length = 5) – không cần xóa.

- **Hủy theo khóa X:** tìm vị trí bằng CD_Search rồi gọi CD_Delete_Element.
- **Mẹo O(1) khi không cần giữ thứ tự:** chép phần tử **cuối** vào vị trí cần xóa: `M[DelPos] = M[Len-1]; Len--;`.

Ví dụ 6.9 – Xóa mọi phần tử bằng X = 5 trong M = (5, 5, 3, 5, 7)

- **Cách sai:**

```
for (int i = 0; i < Len; i++)
   if (M[i] == X)
      CD_Delete_Element(M, Len, i);
```

- Sau khi xóa M[0], phần tử 5 thứ hai dời về vị trí 0 nhưng i đã tăng lên 1 ⇒ **bỏ sót** ⇒ kết quả (5, 3, 7) – sai!
- Thêm nữa mỗi lần xóa dời O(N) ⇒ tổng **O(N²)**.

- **Cách đúng – hai chỉ số, O(N):**

```
void CD_DeleteAll(T M[], int &Len, T X)
{  int k = 0;                  // vị trí ghi
   for (int i = 0; i < Len; i++)
      if (M[i] != X)
         M[k++] = M[i];        // giữ lại
   Len = k;
}
```

i	M[i]	Giữ?	k sau	M[0..k−1]
0	5	Không	0	
1	5	Không	0	
2	3	Có	1	3
3	5	Không	1	3
4	7	Có	2	3 7

- Kết quả **(3, 7)**, Length = 2; mỗi phần tử chỉ được xét **một lần**.

2.8 Sắp xếp, tách, nhập danh sách đặc

- **Sắp xếp:** áp dụng các thuật toán buổi 3–5 trực tiếp trên M[0..Length−1].
- **Tách** danh sách M thành M1 (phần tử thỏa điều kiện) và M2 (còn lại):

```
void CD_Split(T M[], int Len,
              T M1[], int &Len1,
              T M2[], int &Len2)
{  Len1 = Len2 = 0;
   for (int i = 0; i < Len; i++)
      if (DieuKien(M[i])) M1[Len1++] = M[i];
      else                M2[Len2++] = M[i];
}
```

- **Tách đôi** tại vị trí k: M1 = M[0..k−1], M2 = M[k..Len−1].

- **Nhập (ghép nối)** M2 vào cuối M1:

```
int CD_Concat(T M1[], int &Len1, T M2[], int Len2)
{  if (Len1 + Len2 > MaxLen) return 0;   // tràn
   for (int i = 0; i < Len2; i++)
      M1[Len1 + i] = M2[i];
   Len1 += Len2;
   return 1;
}
```

- **Nhập có thứ tự**: dùng hàm **Merge** (buổi 5).
- **Sao chép:** `for (i…) M2[i] = M1[i]; Len2 = Len1;`.

Ví dụ 6.10

M = (15, 8, 22, 3, 9, 30), điều kiện "≥ 10" ⇒ M1 = (15, 22, 30), M2 = (8, 3, 9); nhập M2 vào M1 ⇒ (15, 22, 30, 8, 3, 9).

2.9 Ưu nhược điểm và ứng dụng của danh sách đặc

Thao tác	Độ phức tạp	Nhận xét
Truy cập phần tử thứ i	**O(1)**	Ưu điểm lớn nhất
Tìm kiếm	O(N); **O(log N)** nếu có thứ tự	Tìm nhị phân được
Thêm/hủy ở cuối	O(1)	
Thêm/hủy ở đầu hoặc giữa	O(N)	Phải dời phần tử
Bộ nhớ	Cố định MaxLen	Lãng phí khi ít phần tử; **tràn** khi vượt MaxLen

- **Ưu điểm:** cấu trúc đơn giản, truy cập ngẫu nhiên nhanh, mật độ sử dụng bộ nhớ cao (không tốn con trỏ), dễ sắp xếp và tìm nhị phân.
- **Nhược điểm:** phải xác định trước MaxLen; thêm/hủy giữa danh sách tốn thời gian dời phần tử; khó mở rộng kích thước.
- **Nên dùng khi:** số phần tử ít biến động, biết trước kích thước tối đa, cần truy cập theo chỉ số và tìm kiếm nhiều (danh mục môn học, bảng tra cứu, ma trận).
- **Không nên dùng khi:** thêm/xóa giữa danh sách thường xuyên, kích thước biến động lớn ⇒ dùng **danh sách liên kết** (buổi 7, 8).
- **Cải tiến:** mảng động tự mở rộng (cấp phát lại gấp đôi khi đầy, như `std::vector`).

2.10 Mảng động tự mở rộng – khắc phục giới hạn MaxLen

- **Ý tưởng:** cấp phát động; khi danh sách **đầy** thì cấp phát mảng mới **gấp đôi**, chép dữ liệu sang, giải phóng mảng cũ.

```
typedef struct {
   T  *M;
   int Length;
   int Capacity;        // sức chứa hiện tại
} DynList;
void DL_Init(DynList &L)
{  L.Capacity = 4;  L.Length = 0;
   L.M = new T[L.Capacity];
}
void DL_AddLast(DynList &L, T X)
{  if (L.Length == L.Capacity)       // đầy ⇒ nhân đôi
   {  T *Moi = new T[2 * L.Capacity];
      for (int i = 0; i < L.Length; i++)
         Moi[i] = L.M[i];
      delete[] L.M;
      L.M = Moi;
      L.Capacity *= 2;
   }
   L.M[L.Length++] = X;
}
void DL_Free(DynList &L) { delete[] L.M; L.M = NULL; }
```

Ví dụ 6.11 – Thêm lần lượt 9 phần tử (sức chứa ban đầu 4)

Thêm phần tử thứ	Capacity	Số phần tử phải chép
1 – 4	4	0
5	4 → **8**	4
6 – 8	8	0
9	8 → **16**	8

- Tổng số lần chép = 4 + 8 = 12 < 2·9 ⇒ trung bình mỗi lần thêm cuối chỉ tốn **O(1)** (phân tích **khấu hao**).

- Đây là cách `std::vector` của C++ hoạt động. Vẫn giữ ưu điểm truy cập O(1), nhưng thêm/xóa **giữa** vẫn O(N).
- **Lỗi thường gặp:** quên `delete[]` mảng cũ ⇒ **rò rỉ bộ nhớ**; giữ con trỏ trỏ vào mảng cũ sau khi mở rộng ⇒ con trỏ "treo".

Ví dụ 6.12 – Danh sách SV có thứ tự theo MaSV

```
typedef struct { char MaSV[11]; char HoTen[31]; float DTB; }
SinhVien;
SinhVien DS[MaxLen];  int Len = 0;
// Thêm SV giữ thứ tự MaSV
int ThemSV(SinhVien DS[], int &Len, SinhVien X)
{  if (Len == MaxLen) return -1;
   int i = Len - 1;
   while (i >= 0 && strcmp(DS[i].MaSV, X.MaSV) > 0)
   {  DS[i + 1] = DS[i];  i--;  }
   DS[i + 1] = X;  Len++;
   return i + 1;
}
// Tìm nhị phân theo MaSV
int TimSV(SinhVien DS[], int Len, char *Ma)
{  int F = 0, L = Len - 1;
   while (F <= L)
   {  int Mid = (F + L)/2;
      int c = strcmp(Ma, DS[Mid].MaSV);
      if (c == 0) return Mid;
      if (c < 0) L = Mid - 1; else F = Mid + 1;
   }
   return -1;
}
```

- **Các chức năng khác:**
  - Xóa SV theo mã: TimSV + CD_Delete_Element.
  - Danh sách SV có DTB ≥ 8: duyệt + điều kiện.
  - Tách thành danh sách "đạt" (DTB ≥ 5) và "không đạt".
  - Sắp theo DTB để xếp hạng: sao chép sang mảng phụ rồi sắp (giữ nguyên thứ tự MaSV của danh sách chính).
- **Phân tích chọn CTDL:** lớp ≤ 100 SV, tra cứu theo mã thường xuyên, thêm/xóa ít (đầu học kỳ) ⇒ danh sách đặc có thứ tự là **phù hợp**: tìm O(log N), thêm O(N) nhưng N nhỏ.
- Nếu là danh sách **toàn trường** 30.000 SV, thêm/xóa liên tục ⇒ cân nhắc DSLK hoặc cây nhị phân tìm kiếm (buổi 11).

3.1 Bài thực hành buổi 6 – Thư viện thao tác danh sách đặc

Yêu cầu: xây dựng tập tin CD_LIST.H và chương trình menu

```
#define MaxLen 100
typedef int T;
void CD_Initialize(int &Len);
int  CD_Create_List(T M[], int &Len);
void CD_Show(T M[], int Len);
int  CD_Search(T M[], int Len, T X);
int  CD_Insert_Element(T M[], int &Len, T NewValue, int InsPos);
int  CD_Insert_Order(T M[], int &Len, T X);
int  CD_Delete_Element(T M[], int &Len, int DelPos);
void CD_DeleteAll(T M[], int &Len, T X);
void CD_Sort(T M[], int Len);              // chọn một thuật toán buổi 3-4
void CD_Split(T M[], int Len, T M1[], int &Len1, T M2[], int &Len2);
int  CD_Concat(T M1[], int &Len1, T M2[], int Len2);
```

- Menu: 1. Nhập 2. Xuất 3. Tìm 4. Thêm tại vị trí 5. Thêm có thứ tự 6. Xóa tại vị trí 7. Xóa mọi X 8. Sắp xếp 9. Tách chẵn/lẻ 0. Thoát.
- **Kiểm thử bắt buộc:** thêm khi danh sách **đầy**; xóa khi danh sách **rỗng**; thêm tại vị trí 0, Length, và vị trí không hợp lệ (−1, Length + 1).
- **Mở rộng:** viết lại thư viện cho T = struct SinhVien (MaSV, HoTen, DTB); tìm theo MaSV, sắp theo DTB.

3.2 Câu hỏi và bài tập

Câu hỏi

- 1) Danh sách đặc là gì? Biểu diễn bằng cấu trúc dữ liệu nào? Cần quản lý những thông tin gì?
- 2) Vì sao khi thêm phải dời phần tử **từ cuối về**, còn khi hủy phải dời **từ đầu về sau**?
- 3) Nêu ưu, nhược điểm của danh sách đặc; cho 2 ví dụ nên dùng và 2 ví dụ không nên dùng.

Bài tập (có gợi ý)

- 1) Cho M = (12, 7, 25, 7, 9), Length = 5. Thực hiện liên tiếp: thêm 30 tại vị trí 0; hủy phần tử tại vị trí 3; xóa mọi phần tử bằng 7. → **Đáp số:** (30, 12, 7, 25, 7, 9) → (30, 12, 7, 7, 9) → **(30, 12, 9)**.
- 2) Viết hàm loại bỏ các phần tử **trùng lặp** trong danh sách đặc (giữ lần xuất hiện đầu); đánh giá độ phức tạp. Nếu danh sách đã có thứ tự, làm thế nào để đạt O(N)?
- 3) Viết hàm **đảo ngược** danh sách đặc tại chỗ (không dùng mảng phụ).
- 4) Biểu diễn đa thức bằng danh sách đặc các cặp (hệ số, số mũ) giảm dần theo số mũ; viết hàm cộng hai đa thức bằng thao tác **trộn**.